МИНОБРНАУКИ РОССИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

«Национальный исследовательский университет «МЭИ»

Задание на лабораторную работу №2

по дисциплине «Вычислительные системы»

«Конвейерная модель выполнения инструкций для разработанной ISA»

Цель

Целью лабораторной работы является разработка и реализация конвейерной модели выполнения инструкций для ранее спроектированной вычислительной системы (и ее виртуальной машины), включая формализацию стадий выполнения, обнаружение и разрешение конфликтов, а также количественную оценку эффективности конвейеризации.

Теория

Конвейеризация исполнения инструкций определяется как способ организации вычислений, при котором выполнение каждой инструкции представляется в виде упорядоченной последовательности стадий, а во времени допускается перекрытие выполнения стадий различных инструкций при сохранении семантики последовательного исполнения. Формально, пусть состояние вычислительной системы задаётся набором архитектурных переменных S (включая, как минимум, PC и элементы архитектурного состояния варианта), а семантика каждой инструкции I задаётся частичной функцией перехода $\Delta_I: S \to S$

(частичность отражает возможность ошибок исполнения). Последовательное выполнение программы определяется композицией соответствующих переходов до остановки или ошибки.

Конвейерная реализация задаёт конечный набор стадий K и тактовую динамику, в которой в каждом такте происходит продвижение набора находящихся в исполнении инструкций по стадиям и выполняются операции стадий. Стадия трактуется как отображение, преобразующее промежуточное состояние инструкции (декодированные поля, подготовленные операнды, вычисленные адреса, промежуточные результаты) и, при необходимости, порождающее запрос на изменение архитектурного состояния. Декомпозиция на стадии является частью формальной спецификации исполнения; типовая схема включает IF (выборка), ID (декодирование и подготовка операндов), EX (выполнение и вычисление условий/адресов), MEM (доступ к памяти) и WB (фиксация результата).

Корректность конвейерного исполнения задаётся требованием семантической эквивалентности последовательной модели. В рамках настоящей работы корректность понимается как наблюдательная эквивалентность по архитектурному состоянию: для любых допустимых входных данных и начального состояния S_0 последовательная и конвейерная модели либо завершаются одинаковым конечным архитектурным состоянием, либо завершаются одинаковым видом ошибки, причём момент фиксации результатов инструкций в архитектурное состояние должен быть определён и соблюдён. Практически это достигается введением точки установки (коммита) для каждой инструкции; порядок установки обязан соответствовать программному порядку инструкций, если иное не определено спецификацией.

Конфликты (*hazards*) в конвейере возникают вследствие конкуренции за ресурсы и необходимости соблюдения упорядоченности операций над архитектурным состоянием. Структурный конфликт имеет место, когда в одном такте две или более стадий требуют использования одного и того же ресурса при невозможности совместного обслуживания. Конфликт по управлению связан с тем, что значение PC на этапе выборки зависит от исхода перехода, определяемого на

более поздней стадии, вследствие чего могут быть выбраны инструкции по неверному адресу. Конфликт по данным обусловлен зависимостями между чтением и записью одних и тех же объектов архитектурного состояния различными инструкциями.

Для строгого определения зависимостей используется представление инструкции через множества доступа к архитектурным объектам. Для каждой инструкции I определяются множества $R(I)$ (читаемые объекты) и $W(I)$ (записываемые объекты). Для двух инструкций I_a и I_b, расположенных в программном порядке a < b, вводятся зависимости:

- RAW (Read After Write) имеет место, если $W(I_a)$ пересекается с $R(I_b)$ и чтение в I_b должно наблюдать результат записи Ia;
- WAR (Write After Read) имеет место, если $R(I_a)$ пересекается с $W(I_b)$ и запись I_b не должна опережать чтение I_a;
- WAW (Write After Write) имеет место, если $W(I_a)$ пересекается с $W(I_b)$ и порядок записей обязан совпадать с программным.

В конвейерной модели зависимость проявляется как необходимость ограничить продвижение инструкции, если требуемое значение ещё не сформировано или ещё не зафиксировано в архитектурном состоянии.

Стратегии разрешения конфликтов формализуются как правила управления продвижением инструкций по стадиям и, при наличии, правила маршрутизации результатов. Базовой стратегией является интерлокинг (*stall*): при обнаружении конфликта продвижение одной или нескольких инструкций задерживается на один или более тактов до устранения причины конфликта; при этом в конвейере фактически формируется пузырь (*bubble*), то есть пустая позиция стадии. Для конфликтов по управлению применяется очистка (*flush*): инструкции, выбранные по неверному PC, аннулируются до установки, а PC устанавливается в целевое значение; тем самым сохраняется наблюдательная эквивалентность последовательной модели.

Алгоритм разрешения RAW-конфликтов в тактовом симуляторе задаётся сравнением наборов чтения и записи с учётом стадий формирования и фиксации результата. Для каждой инструкции должны быть определены

(i) момент, когда операнд читается,

(ii) момент, когда результат становится доступным, и

(iii) момент фиксации результата в архитектурном состоянии.

В базовой модели без пересылки (forwarding) результат считается доступным только после фиксации, поэтому конфликт устраняется задержкой младшей инструкции до завершения фиксации старшей. При наличии пересылки результат допускается использовать на более ранней стадии (например, из EX или MEM), что уменьшает число тактов простоя; правила пересылки должны быть формально определены и не должны изменять архитектурную семантику.

Алгоритм разрешения структурных конфликтов задаётся моделированием ресурсов и их занятости. Каждый ресурс описывается пропускной способностью и правилом совместимости запросов в одном такте. Конфликт фиксируется, если число несовместимых запросов превышает пропускную способность; разрешение выполняется детерминированной задержкой одной из конкурирующих инструкций (как правило, с приоритетом старших инструкций).

Критерии эффективности конвейера определяются в тактовой модели исполнения. К базовым показателям относятся число тактов C до завершения, число выполненных инструкций N, среднее значение $CPI = {}^{C}/_{N}$, ускорение относительно последовательной модели $S_{\text{уск}} = \frac{C_{\text{посл}}}{C_{\text{конв}}}$, а также разложение C на составляющие потерь: такты простоя из-за зависимостей по данным, такты простоя из-за структурных конфликтов и такты, потерянные на очистку при переходах. Эффективность конвейера считается тем выше, чем меньше CPI при сохранении корректности и чем меньше доля тактов, не приводящих к установке полезной инструкции.

Задание

Требуется расширить спецификацию ISA формальными сведениями, необходимыми для конвейерного исполнения, реализовать симулятор конвейера с тактовой моделью и механизмами разрешения конфликтов, а также провести сравнительный расчёт характеристик выполнения для последовательной и конвейерной моделей на заданной программе.

Модель конвейера

Конвейер должен быть реализован в виде тактового симулятора, в котором в каждом такте выполняется одновременное продвижение инструкций по стадиям и, при необходимости, вставка задержек/очисток. Конвейерная модель должна быть детерминированной.

В работе используется пятистадийная схема (или эквивалентная по смыслу, если студент формально обоснует отличия): IF (выборка), ID (декодирование и подготовка операндов), EX (выполнение/вычисление адресов/условий), MEM (доступ к памяти, если применимо), WB (фиксация результатов в архитектурном состоянии).

В базовой модели допускается упрощение, согласно которому все арифметические операции имеют одинаковую латентность и занимают одну стадию EX продолжительностью в один такт. Различия во временных характеристиках отдельных операций (например, умножения или деления) могут вводиться только в расширенных вариантах реализации и рассматриваются как элемент микроархитектурного усложнения, не влияющий на архитектурную семантику ISA.

Момент изменения архитектурного состояния должен быть формально определён. Рекомендуется фиксировать все архитектурные записи на стадии WB; для инструкций перехода момент изменения PC (принятие решения о переходе) должен быть задан и соблюдён (например, EX).

Требования к расширению спецификации ISA

Спецификация каждой инструкции должна включать сведения, достаточные для построения автомата обнаружения конфликтов. Для каждой инструкции необходимо формально задать: множество читаемых объектов архитектурного состояния, множество записываемых объектов архитектурного состояния, стадию, на которой формируется результат, стадию, на которой происходит запись результата, а также признак влияния на поток управления (изменение PC) и стадию разрешения перехода.

Под «объектами архитектурного состояния» понимаются элементы состояния, определённые вариантом архитектуры (регистры, ACC, стек/указатель стека, память данных, флаги состояния, PC). Если разработчик допускает запись в память как часть инструкции, данная запись должна быть отражена в множестве записей и в моменте фиксации.

Требования к реализации

Интерпретатор должен поддерживать два режима выполнения: последовательный (эталонный) и конвейерный. Последовательный режим обязан сохраняться как проверочная модель корректности.

Конвейерный режим обязан реализовывать: обнаружение конфликтов по данным (минимум RAW), обнаружение структурных конфликтов (минимум: ограничение на доступ к памяти данных как к единственному ресурсу на стадии MEM, если модель это предполагает), а также конфликты по управлению для инструкций условного и безусловного перехода.

Разрешение конфликтов в базовом варианте должно выполняться путём stall и/или bubble таким образом, чтобы результирующее архитектурное состояние совпадало с последовательным выполнением. Для переходов должна быть реализована очистка конвейера для инструкций, находящихся «младше» принятого перехода, если они были выбраны по неверному значению PC.

Реализация должна обеспечивать сбор статистики: общее число тактов, число выполненных инструкций, CPI, количество остановов конвейера из-за зависимостей по данным, количество остановов из-за структурных конфликтов и количество очисток из-за переходов.

Должен быть реализован отладочный режим, позволяющий по тактам наблюдать состав стадий конвейера и изменения архитектурного состояния. Наличие отладочного режима не должно влиять на семантику.

Варианты

В качестве тестовой программы используется один из алгоритмических вариантов. Необходимо обеспечить, чтобы входные данные и размещение результатов в памяти были определены в спецификации и неизменны между режимами.

Требуется выполнить программу в последовательном и конвейерном режимах и представить сравнительные показатели (такты, CPI, число простоев/очисток). При наличии отличий результатов вычислений последовательного и конвейерного режимов работа считается невыполненной.

Так же студенту/бригаде назначается один уровень реализации конвейера.

A. базовый конвейер с разрешением конфликтов методом stall/flush, учёт RAW-зависимостей и конфликтов по управлению.
B. конвейер с пересылкой результатов для устранения части RAW-зависимостей; правила пересылки должны быть формально определены и реализованы.
C. конвейер с поддержкой многотактных операций (например, умножение/деление) с моделированием занятости стадии EX; влияние на CPI должно быть отражено в статистике.
D. конвейер с разделением класса инструкций по ресурсам (например, отдельные функциональные блоки для ALU и MEM) и моделированием структурных конфликтов на основе занятости ресурсов.

Задачи:

1. Вычисление факториала числа N.
2. Вычисление N-го числа последовательности Фибоначчи (итерационная форма).
3. Вычисление наибольшего общего делителя двух чисел (алгоритм Евклида).
4. Определение максимального элемента в массиве фиксированной длины.
5. Определение минимального элемента в массиве фиксированной длины.
6. Вычисление суммы элементов массива.
7. Вычисление среднего арифметического элементов массива (целочисленное деление).
8. Подсчёт количества положительных элементов в массиве.
9. Проверка числа N на простоту (делением на последовательность делителей).
10. Возведение числа в натуральную степень (итерационным умножением).
11. Реверс массива (перестановка элементов в памяти).
12. Сортировка массива методом выбора или обмена (любой простой квадратичный алгоритм).
13. Поиск элемента в массиве (линейный поиск, возврат индекса либо специального значения при отсутствии).
14. Вычисление суммы чисел от 1 до N с использованием цикла.
15. Вычисление значения полинома по схеме Горнера для заданного массива коэффициентов.

Номер в группе (или бригада)	**А-07**		**А-08**		**А-12**		**А-17**	
	Конвейер	Задача	Конвейер	Задача	Конвейер	Задача	Конвейер	Задача
1	A	3	B	7	C	12	D	14
2	B	4	C	8	D	13	A	15
3	C	5	D	9	A	14	B	1
4	D	6	A	10	B	15	C	2
5	A	7	B	11	C	1	D	3
6	B	8	C	12	D	2	A	4
7	C	9	D	13	A	3	B	5

8	D	10	A	14	B	4	C	6
9	A	11	B	15	C	5	D	7
10	B	12	C	1	D	6	A	8
11	C	13	D	2	A	7	B	9
12	D	14	A	3	B	8	C	10
13	A	15	B	4	C	9	D	11
14	B	1	C	5	D	10	A	12
15	C	2	D	6	A	11	B	13

Контрольные вопросы

1. Сформулируйте условия корректности конвейерного исполнения относительно последовательной модели.
2. Определите классы конфликтов (структурные, по данным, по управлению) и приведите их формальные признаки в терминах читаемых/записываемых множеств.
3. В чём состоит различие между stall, bubble и flush? Как эти механизмы влияют на статистику CPI?
4. Почему для конвейерного исполнения требуется фиксировать момент записи результата в архитектурное состояние?
5. Каким образом определяется RAW-зависимость для двух инструкций в конвейере?
6. На какой стадии целесообразно разрешать условный переход и почему? Как выбор стадии влияет на потери от конфликтов управления?
7. Почему даже при идеальном конвейере CPI не обязан стремиться к 1 на произвольной программе?
8. Как изменится модель конфликтов при переходе от аккумуляторной архитектуры к регистровой?
9. Какие ограничения на доступ к памяти приводят к структурным конфликтам, и как они моделируются в тактовом симуляторе?
10. Какие признаки в спецификации ISA являются минимально достаточными для реализации конвейерного симулятора?